



Concurrent-Development Process Model

MIKIO AOYAMA, *Fujitsu*

◆ *This model lets you develop multiple functions concurrently, from requirements specification to system test. It has been used to develop a large-scale communication system.*

Shortening the development cycle has become a major goal in software development, yet few conventional process models address this issue. In an effort to shorten the cycle, developers are looking at ways to alter the development process from sequential to concurrent. As this trend continues, the need for a concurrent process model is growing.

In the mid 1980s, we at Fujitsu created a model that lets you develop multiple functions concurrently over the entire development process — from requirements specification to system test.¹ For the last five years, we have applied the concurrent-development model to the development of a large-scale communication system and have produced more than one million lines of source code. A similar system developed with a conventional, sequential process model had a development cycle of one year. Our development cycle was three months.

The complexity of concurrent devel-

opment forced us to add new process-management techniques and to reengineer conventional ones. We drew much of the model's underlying philosophy from the principles of continuous process improvement found in Japanese production systems for hardware assembly.

Shortening the development cycle was an important goal of the project, because communication systems are rapidly evolving to provide hundreds of new features such as Integrated Services of Digital Networks and computer-communication-integrated services.

However, our initial goal expanded to a reengineering of the entire development process. Part of the reason is that a short development cycle makes the project riskier but helps expose potential problems in the development process itself.

We found, for example, that we were forced to manage the process more carefully. To control the process, as well as the

products and project, we implemented an integrated management system consisting of a broad spectrum of management techniques. Besides cutting cycle time, this management system let us use a small, fixed number of developers. This, in turn, stabilized fluctuations in workload and development size — both of which are key to managing large-scale projects.

In this article, I describe some of the lessons learned in managing the concurrent model, focusing on the management of the concurrent process itself and touching briefly on organizational issues.

EVOLUTIONARY CHANGE

Figure 1 shows the two-dimensional nature of the changes in software development. Along one axis, the process is moving from sequential to concurrent; along the other, from centralized to distributed. Within these axes are four types of process model:

- ◆ *Centralized, sequential.* This model reflects the conventional development process. Development teams are located within a single organization or at a single site, and development proceeds sequentially from requirements specification to system test. This model can be widely applied to developing large-scale systems, from business-data-processing to real-time, mission-critical systems. Its drawbacks are a long development cycle and difficulty in allocating large numbers of developers in a single site.

- ◆ *Centralized, concurrent.* Development teams are still within the same organization or at the same site, but teams are developing multiple features concurrently. While this model can shorten the development cycle, it creates a new, complex problem: how to coordinate multiple concurrent activities.

- ◆ *Distributed, sequential.* Development is split among teams that are not located within the same organization. Typically, this occurs when some part of the process, like coding, is contracted to multiple software houses. Although process management is not as complex as that in concurrent development, resource distribution causes many problems. It is costly to

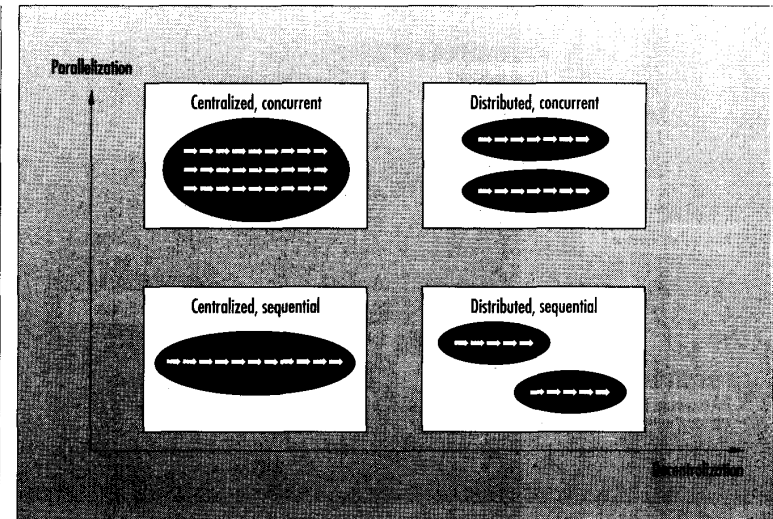


Figure 1. Two-dimensional evolution from centralized, sequential development to distributed, concurrent development.

collect and disseminate information among distributed development teams. Dividing the process into multiple teams may block the smooth flow of information and development knowledge.

- ◆ *Distributed, concurrent.* Multiple development organizations are concurrently developing multiple features of a system. This is the ultimate model for downsizing the development process and organization. The complexity is almost double that of any other model because you are combining distribution and concurrency issues, but the benefits are well worth the trouble. The shorter development cycle helps expose potential problems in the development process, which can lead to more reliable, cost-effective products.

This evolution, which affects everything from individual developers to entire projects, caused us to rethink the development process.

We evolved from a centralized, sequential

model first to a centralized, concurrent model, and finally to a distributed concurrent model. The evolutionary path depends largely on where the development effort is to begin with. For example, our first transition was to concurrency, since

our initial goal was to shorten the development cycle. However, the concurrent process is much more complex than a traditional process, so we also learned much about process management. Our work in developing and verifying new management techniques helped us be better prepared for the second transition, to a distributed concurrent model. For example, we were able to

use our process-definition and redesign techniques in maintaining a common process baseline among geographically distributed development teams. Everyone spoke the same development language, and maintained a consistent level of quality in both the development work and the resulting products.

**WHAT
EVOLUTIONARY
PATH YOU
TAKE DEPENDS
LARGELY ON
WHERE YOU
ARE TO BEGIN
WITH.**

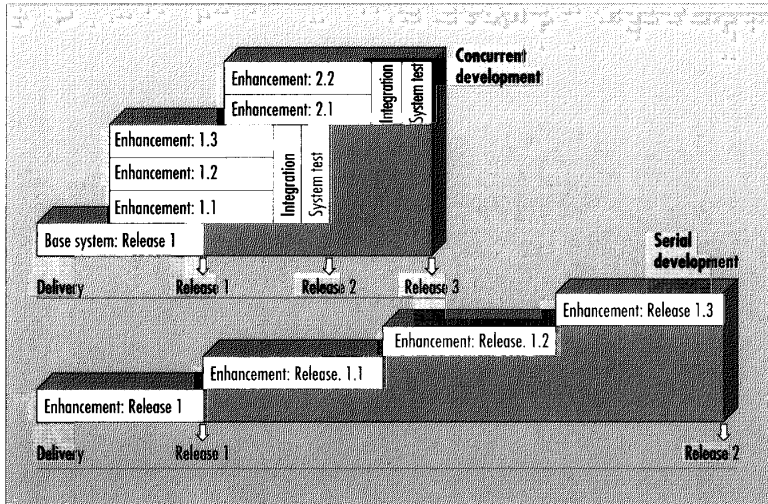


Figure 2. Serial development versus concurrent development. In serial development, an enhancement can be only a function; in concurrent development it can be a unit that comprises multiple functions or modules. Both systems shown are evolving, but at different speeds. In concurrent development, we can deliver multiple functions in less time because enhancements are being done with pipelined processes and cyclic enactment, not with a serial process. Consequently, integration and system test can be done earlier, and there is less time between releases.

CONCURRENT DEVELOPMENT

Figure 2 shows the concurrent development model, which differs from conventional process models in three ways:

- ◆ *Concurrency is in the entire process.*

Even conventional sequential process models have some process entities that can be executed concurrently. In our model, however, the entire development process can be executed concurrently.

- ◆ *It allows "process in the large."* Various process models — from the waterfall and spiral models to the new paradigms² and other varieties³ — have been proposed. However, these traditional models focus on a single sequence of software-development processes; concurrent development focuses on the concurrent execution of multiple processes, just as programming in the large focuses on the execution of multiple programs.

Moreover, because conventional models include some concurrent entities, developers face many of the same problems in concurrently executing processes that they face in concurrent development.

- ◆ *Concurrency is coarse-grained.* In sequential models, some program modules are concurrently implemented at lower development levels, where the concurrency unit is at the module, or fine-grained, level. In concurrent development, the concurrency unit can be a feature that comprises multiple functions or modules. The developer must therefore deal with how processes and products

interact at higher levels, which most traditional models ignore.

CHALLENGES

The structure and dynamic behavior of a concurrent development process are

much more complex than in conventional models. We found some issues to be similar to those encountered in the production process of Japanese hardware systems, which emphasize continuous process improvement.^{4,5} The heart of these production systems is the careful orchestration of various management techniques for the production process. Although we certainly see differences between them and software development, we have borrowed some of their underlying philosophies.

In process management, we identified several challenges stemming directly from the nature of concurrent development. The most significant is the complicated interaction among concurrent processes and products that may be updated concurrently. Figure 3 shows this complex interaction. An *enhancement* is a unit that comprises multiple functions or modules. Multiple enhancements can be developed concurrently and integrated as a system, then delivered to customers as a release.

Process interactions. The developer must consider two types of process interactions in concurrent development: multiple-enhancement and multiple-release. The development of a release involves multiple enhancements that may interact each other. The delay of an enhancement may affect other enhancements and may delay the entire release.

In multiple-release interactions, the delay of a release may require rescheduling the development of successive releases. If the delay exceeds some threshold absorbed by successive releases, the cumulative delay can jeopardize the project.

Product interactions. Multiple teams working on related enhancements may disrupt the system's continuity. In requirements specifications, for example, this can cause inconsistent or incomplete specifications. In design and implementation, simultaneous updates to a single module may violate that module's consistency.

These problems are familiar in the design of concurrent systems, but interactions among requirements specifications are at a higher level than those encoun-

**WE BORROWED
HEAVILY
FROM THE
PHILOSOPHIES
OF LEAN
DEVELOPMENT
SYSTEMS.**

tered later during design and implementation. In higher level development processes, interactions are intangible and are created as specifications are elaborated. Furthermore, in integration and system test, we must ensure that multiple releases developed concurrently have not violated the consistency of the entire system.

IMPLEMENTATION MANAGEMENT

To manage concurrent development, we created the system shown in Figure 4,

which consists of five main elements, integrating a range of management techniques together with a support environment. Elements in this system are not new in and of themselves, but their implementation is novel because we are applying them to a concurrent-development scheme. We had to invent new management techniques and reengineer old ones. The focus of this article is on process management, although I briefly present the other four elements to give a picture of the complexity we faced:

♦ *Process management.* Process management deals with defining and instantiating development processes, assigning development teams, and controlling the execution. The developer must visualize the process and its interactions and decouple them, which means restructuring the process model's underlying concurrent-development process. To accommodate dynamic behavior, process control must reach every level of the process structure, from entire project to release and enhancement as well as from team to indi-

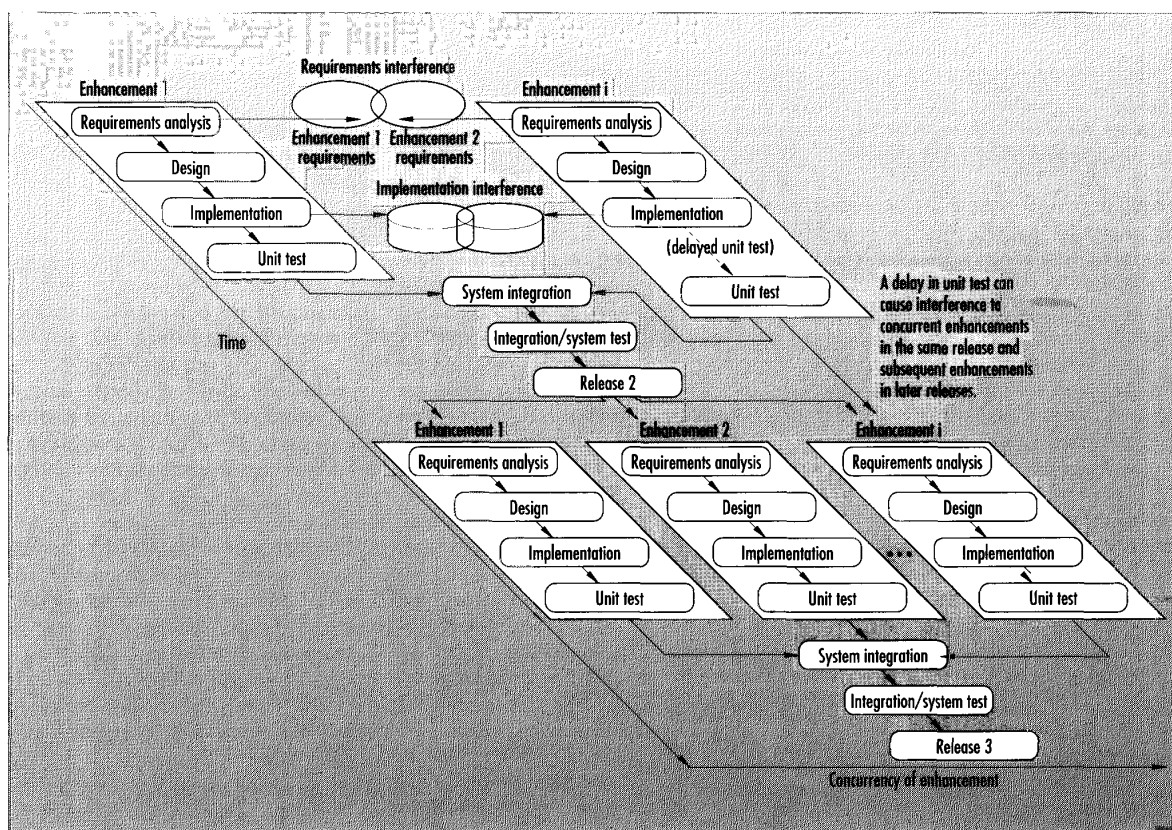


Figure 3. Diagram of the interactions in the concurrent-development process model, which shows the complexity of interacting enhancements and releases. Interactions are either indirect or direct. Examples of indirect interaction are conflicting enhancement requirements and implementation consistency (upper center). Direct interactions include the interference of individual requirements and implementation issues along the concurrent-development path of enhancement 1 ... enhancement i . For example, a delay (delays are denoted with thick, black arrows) in enhancement i 's unit test (upper right corner) may delay the unit test of other enhancements in the same release. Thus, the integration test must be rescheduled, which, in turn, may affect the entire development schedule of release 2 and delay the start of development on release 3. Moreover, if an enhancement in a subsequent release is sufficiently major, it may be under development concurrently with the previous release. In the figure, the delayed unit test of release 2 has delayed enhancement j in release 3. The main task in managing concurrent development is to decouple these interactions to avoid conflicts and inconsistency. In the bottom third of the figure, interactions have been successfully decoupled, so interference is at a minimum.

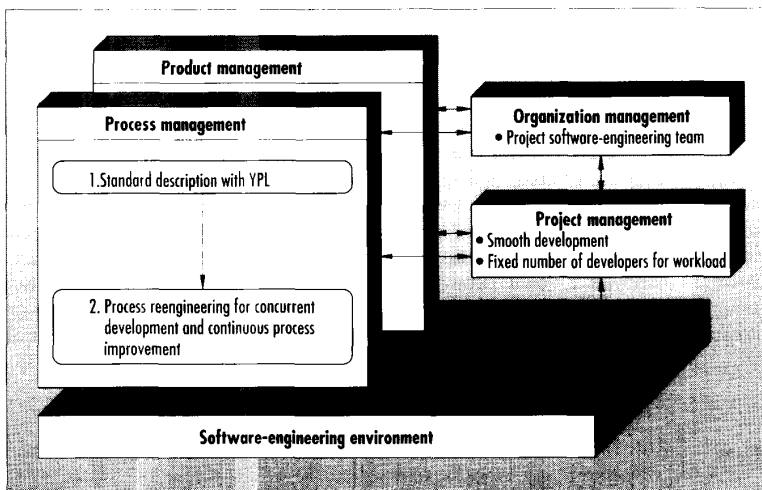


Figure 4. Framework of the management system for concurrent development.

vidual developer. For the large-scale communications project, therefore, we described the entire development process up to the details of the individual developer's task, and analyzed them. We then restructured a conventional process model to support concurrent development and developed a process-management system. I describe our work in more detail in the next section.

◆ **Project management.** Project management must efficiently assign and control development resources to ensure productivity and quality over multiple releases. From an economical point of view, the number of developers involved in the projects should be fixed because rapidly changing developers costs almost every aspect of development. To cope with these problems, we developed a cyclic process-enaction model that can iteratively assign a fixed number of developers from one development process to another.

◆ **Product management.** To manage products along with concurrent development processes, the developer must con-

sider how products differ according to the processes that produced them. We separated products into two categories: tangible and intangible. In requirements specification and design, products are intangible and concurrently elaborated by multiple teams. The interaction in this process is difficult to manage automati-

cally, so we have taken the review-and-inspection approach together with process control and organizational support. In lower level processes, an integrated configuration-management system plays the central role in managing tangible products.

◆ **Organization management.** To implement concurrent development, we established a Project Software Engineering Team within the development project. Cooperating with developers and research laboratories, the PSET's major roles were to define the development process; manage the products; coordinate the software-integration process; develop and implement management techniques; plan, specify, and develop software-engineering environments; and

coordinate with research labs and other software-engineering groups in the introduction of new technologies. The PSET also works as a management-support center by collecting and distributing various statistics and consulting each development group.

The main advantage to having the PSET was that they, as project insiders, could understand real needs in managing concurrent development and bring the latest practical technology to the development teams.

◆ **Software-engineering environment.** We developed an integrated software-engineering environment that is indispensable to implementing and operating the concurrent-development system in large-scale projects. The environment provides a balanced view to process, project, and product management and support management techniques as a system.

PROCESS MANAGEMENT

Concurrent-development processes have two distinct process dynamics. First, because you must manage the processes along with the process definition, you must be able to decouple the interaction in the development activities of multiple teams. Second, you must manage a variety of events that happen arbitrarily and enable many people to share information about these events.

Because none of the traditional process models accommodate these dynamics, we decided to adapt one to concurrent development. We chose the waterfall model because, although it has been the subject of much debate, it is still widely used, and it seemed a well-defined starting point for evolution to concurrent development. We re-engineered it to fit concurrent development through the iteration of three processes:

1. Define the process.
2. Redesign the process structure.
3. Redesign process enaction.

Defining the process. We used a process model to define the highly complex structure and dynamic behavior of the concurrent development process. As part of our model, we developed a visual process de-

MOST MODELS IGNORE HOW TO DEAL WITH PROCESS AND PRODUCT INTERACTION AT THE HIGHER LEVELS.

scription language, YPL (YAC II-oriented Process-Description Language), which is an extension of the YAC tree-structured chart.^{6,7} Figure 5 gives a sample of YPL notation. YPL let us use the same language for the development process that was used to describe objects in the communications system. We have defined our entire development process — from project planning, to requirements specification to delivery in YPL.

As we continually reengineered the defined process, the PSET coordinated input from the development teams to help elaborate the process definition. We used the definition in many ways, including to analyze development cost.

Redesign process structure. Our goal in redesigning the structure was to fit the process to concurrent execution as well as cut redundancy. A key part of this was to control the synchronization of concurrency, especially in the middle of the cycle. To satisfy our goal, we incorporated several mechanisms:

◆ *Process-dependent management view.* The software-integration process separates the entire development process into two subprocess categories, upper and lower. The upper subprocess category covers requirements specification to pre-integration test. The lower subprocesses cover integration test to product delivery. The two categories have different management requirements. The management of upper subprocesses focuses on decoupling interactions among multiple teams; the management of lower subprocesses focuses on the quality assurance of the single integrated system.

◆ *Decoupling of interactions.* One of the major drawbacks of the waterfall model is that identifying problems in requirements specifications is often postponed until the very end of the development process. In concurrent development, this drawback is more serious because the coupling of multiple functions or enhancements is more complex. Furthermore, this coupling causes interference among development teams. To avoid these problems, we have added a series of reviews, which take place in the

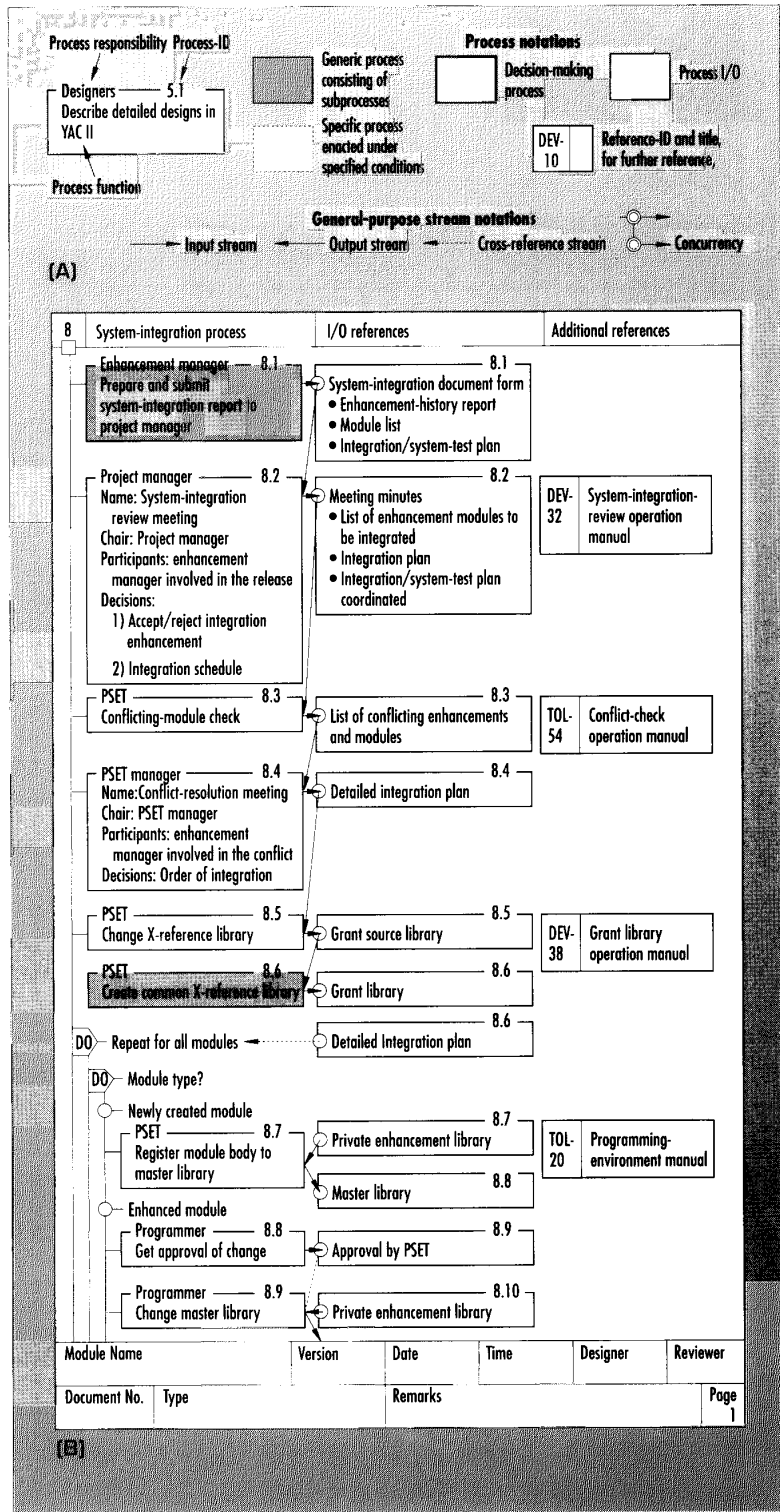


Figure 5. (A) Graphical notation of YPL, the language used to describe the entire development process. Developers can use YPL to define both the development process and objects in the communication system being developed. (B) YPL description of part of a process, illustrating some of the YPL notations in (A). The left part of the box defines the procedure, the center defines the dataflow (procedure inputs and outputs), and the right side defines the further reference to the procedure.

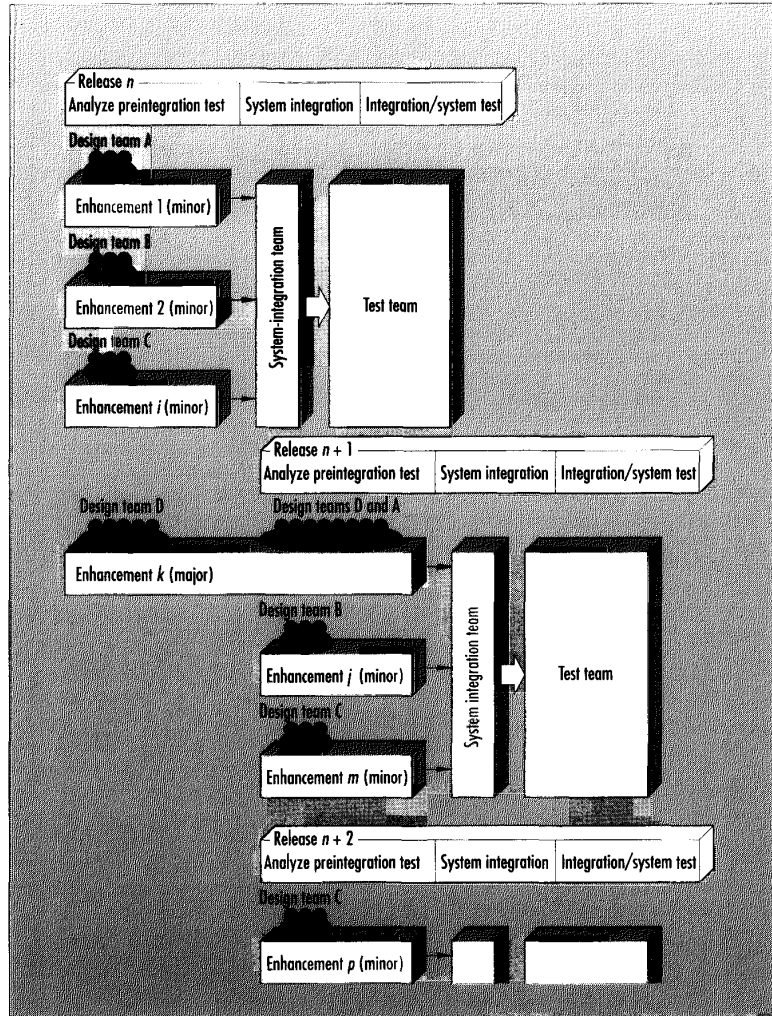
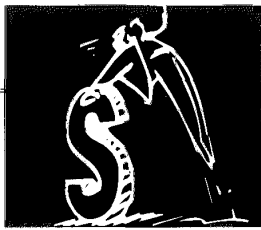


Figure 6. A model showing the cyclic enaction of the concurrent-development process. Teams A, B, and C (whose members are denoted by a circle atop a triangle) are working on enhancements 1, 2, and 3, respectively. When preintegration tests have been completed, these enhancements are integrated into release n and passed to the test team. The development teams move on to the development of the next release, which consists of enhancements k , j , and m . Because enhancement k is a major enhancement in the release of a large-scale feature, its development period is double that of the minor enhancements (j and m). Therefore, team D initiates the development of enhancement k the same time they initiate release n . The number of developers is theoretically unchanged over multiple releases, from n through $n+1$, and so on, but in this case, Team D joins team A.

upper subprocesses. After each team internally reviews the specifications and design, multiple teams must conduct intensive interteam reviews to identify the expected influence of the function or enhancement on later parts of the development process.

♦ **Software integration.** This is a key point in the synchronization of development processes and products with multiple enhancements. During this step, multiple enhancements, concurrently developed by multiple teams, are incre-

mentally integrated into a single system. After intensively reviewing the previous development processes and quality levels, the products, modules, and documents go into a project repository under the control of PSET. The process is also used to resolve conflicting updates.

♦ **Preintegration testing.** This step ensures the conformance of the interface between the baseline system and the changed or newly created modules. It occurs before software integration.

♦ **Concurrent testing.** During integration test and system test, multiple replicas of integrated software are concurrently tested by multiple test teams that are geographically distributed. Although each enhancement must pass a preintegration test, we focus on conformance among multiple functions. We also added a mechanism to support the management of this testing.

Redesign process enaction. Managing process execution, or enaction, has two main goals. The first is to assign the workload to multiple teams who are concurrently developing multiple enhancements. The second is to control the workload over multiple releases.

To meet these goals, we developed a cyclic-enaction model, shown in Figure 6, and supporting management techniques. The model's core idea is that the development of each enhancement must be completed within a fixed time and iterated over multiple releases. The enaction model lets us develop multiple features with a fixed workload because a fixed number of developers moves from one enhancement to another. However, it raises another challenge — how to precisely control process, project, and products over multiple releases when each process instance must meet an exact development schedule (just-in-time management).

The cyclic-enaction model used a period of six months as the development time, with a release being delivered at the end of three months and at the end of the six months. In the first three months, the teams completed the upper subprocesses; in the next three months, the products went through the lower subprocesses.

We needed supporting project-management techniques because although we could fix the total workload theoretically, for large-scale development, such a constraint is not realistic for managing projects safely. We had to allow for some workload fluctuations, and hence some productivity loss, which the techniques are designed to overcome.

Also theoretically, we could shorten delivery time by shortening the development cycle. In our model, however, the development cycle can be no shorter than the test period (three months); a shorter cycle could sacrifice both productivity and quality.

EXPERIENCE

From applying the model to a large-scale communications system over the last five years, we have learned much about how to improve the development process. What we have learned can be viewed in many contexts.

Comparison to sequential development. In another project, we used a conventional process model to develop a communications system with an architecture similar to that of the large-scale system to which we applied the concurrent-development model. In the following description, S_1 represents the system developed with a sequential process; S_2 is the system developed with the concurrent process.

There are several significant points of comparison:

- ◆ *Length of development cycle.* The release cycle of S_1 was one year; the release cycle of S_2 was three months — a 75-percent reduction.

- ◆ *Ease of construction.* Unlike the sequential-development model, the concurrent-development model let us divide the large-scale software into small subsystems and incrementally construct the whole system. Also unlike a sequential approach, concurrent development provides practical management techniques to incrementally deliver the large-scale software system in a short and fixed interval. This let us get quick feedback from customers on the releases delivered.

- ◆ *Smoothness of project management.*

TABLE 1
STANDARD DEVIATION OF RELEASE SIZE AND LOAD
FOR A SYSTEM DEVELOPED CONCURRENTLY (S_2)

	Number in sample	Standard deviation
Major release	9	21.1%
Minor release	9	20.5%
Workload (six-month cycle)	11	11.7%

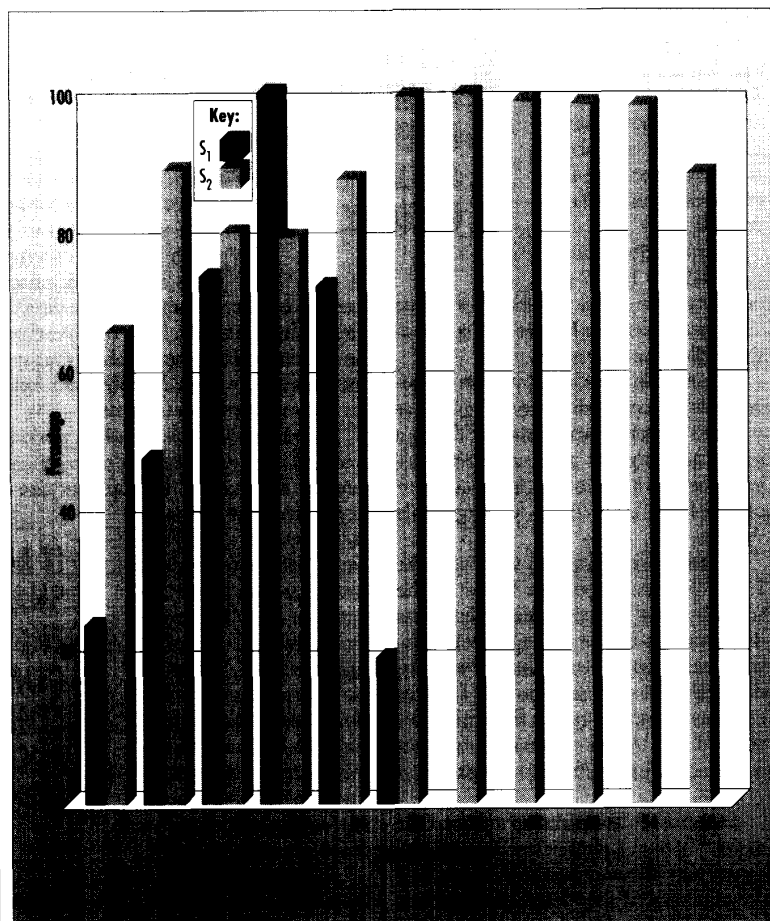
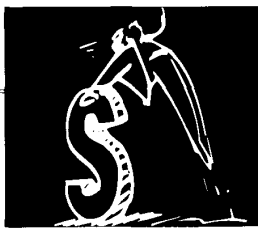


Figure 7. Development workload for two communication systems with similar architectures. S_1 was developed with a sequential process; S_2 was developed with a concurrent process. The development of S_1 is almost complete after 30 months; the development of S_2 is longer and still evolving. Although the peak workload of S_2 is actually four times that of S_1 , for comparison, the figure shows the workloads normalized with peak values. Fluctuations in workload are due to both size and number of developers. The fluctuations are much less for S_2 , primarily because the concurrent model lets you fix the number of developers from enhancement to enhancement and provides the cyclic-enaction model, which helps recover productivity loss from workload fluctuations.



Concurrent development made project management smoother. Figure 7 shows the development workload — which includes changes in the number of developers — for both S_1 and S_2 . While the workload for developing S_1 follows the conventional Railey curve proposed by Lawrence Putnum and colleagues,⁸ that of S_2 is almost fixed.

We evaluated the fluctuation of development size and workload for S_2 in terms of the standard deviation from the average. Table 1 gives the size fluctuations in nine major and nine minor enhancements. As the table shows, the fluctuations are relatively small and well controlled.

The workload fluctuation in Table 1, 11.7 percent, is also very low compared with the size fluctuation. We believe this is due to the cyclic-enaction model.

Changes in workload (number of developers) causes productivity loss as well as stress to corporate management. This is a strong argument for using concurrent development (which lets you fix the number of developers), especially for large-scale systems.

In addition to size and workload, the PSET was an important factor in the smoothness of project management. The development of the management techniques and support environment was neither quick nor painless. The PSET had to elaborate the techniques as the project developed, and this continuous improvement made a significant difference long term.

Comparison to lean production systems. Our system of management techniques for concurrent development shares some of the concepts of the "lean" development system developed and implemented by Tai-ichi Ohno and colleagues of Toyota.^{4,5} The lean production system — which gets its name from continuous-improvement techniques that eliminate redundancy (fat) from the production process — integrates a variety of production management tech-

niques into a system. At the heart of the system are the five major concepts:

- ◆ smooth production by narrowing the fluctuation of production size and precisely managing the flow of products,
- ◆ standardization of work,
- ◆ flexible manufacturing to enable quick changes on the production line,
- ◆ continuous process improvement, and
- ◆ intelligent automation.

In the development and implementation of the concurrent-development process, we found a treasure of management techniques in these principles. Although the techniques are slanted toward hardware assembly, which is rather different from software development, we believe many ideas are directly applicable because the focus is on continuous process improvement. Taking the same five concepts, we can identify much relevance to concurrent development:

◆ **Smooth production.** This idea works as a central principle of project management because a consistent development size is key to cyclic process execution within a fixed period. Software development has one major advantage over hardware assembly: the control of products flow. In just-in-time hardware production, managers must have a way to eliminate inventory. In software development, any products can be transmitted through a communication network.

◆ **Standardization of work.** This is a basic principle in producing any standardized product. The production system provides mechanisms not only to standardize the work process but also to predict performance, which may be difficult in current development practices. Additional study of development processes and metrics will help this problem.

◆ **Flexible manufacturing system.** This property is somewhat specific to hardware assembly, and is bound by the physical constraints of tools and products. We may need this principle in the future, but so far, we have not been emphasizing it in software development.

◆ **Continuous process improvement.** This is extremely important in software development, as already described

◆ **Intelligent automation.** The production system puts more emphasis on process than automation, but if it is automated, the system must be intelligent so that it can stop automatically when problems occur. We can implement this principle somewhat in the management techniques and development environment, but it is still difficult to automate the higher level development processes. Although many computer-aided software-engineering environments have been developed, little attention has been paid to this issue.

Comparison to the software factory. We believe concurrent development is an advanced form of the software-factory approach. For example, our model comprises all nine of the elements Michael Cusumano identified in a software factory,⁹ which can be categorized in four groups:

- ◆ Work and skill standardization
- ◆ Continuous process improvement by commitment to process improvement, a product-process focus and segmentation, process-quality analysis and control, tailored and centralized process R&D, dynamic standardization, and incremental product

improvement

- ◆ Computer-aided tools and integration
- ◆ Systematic reusability

The first three groups are common to lean production and concurrent development. Although not directly related, software reuse is also practiced in concurrent

OUR MODEL COMPRISES ALL THE ELEMENTS ESSENTIAL TO A SOFTWARE FACTORY.

IT ALSO SHARES THE CMM'S GOAL OF CONTINUOUS PROCESS IMPROVEMENT.

development. Concurrent development adds another requirement to software factories. The conventional factory approach focuses on productivity and quality with little attention to the development cycle and the total cost over multiple cycles. Concurrent development's systematic way of developing software systems with a fixed number of developers enhances productivity and quality. The need for a shorter development cycle is also receiving attention in the software factory.

Comparison to CMMs. The Capability Maturity Model, developed by the Software Engineering Institute,¹⁰ and our development process share the same goal: continuous improvement of the software process.

The CMM defines five maturity levels. To implement the concurrent development system, we had to have a well-defined and mature underlying software process — at least level 2, the repeatable level in the CMM. It is widely recognized in hardware production that just-in-time production helps uncover potential process problems that might not be exposed in conventional production systems. We experienced the same effect in implementing concurrent development. We were able to discover many weak points in the development process, management, and development environments through a concurrent-development model. Solving each problem required patience, but the result was a much-improved development process and environment.

Although our initial goal in implementing the concurrent-development model was to shorten the development cycle, we ended up reengineering the entire development process. The problems we identified open up a challenge to managing the flexible and economical development of large-scale software systems.

Several areas warrant future study to

refine the model and management techniques both theoretically and practically.

WE ENDED UP ENGINEERING AN ENTIRELY NEW PROCESS.

From a process-engineering point of view, we should further study modeling techniques for concurrent development. YPL can model the procedural view of the process and define the start and end of concurrency, but it is still weak in modeling multiple views of processes — functional, structural, and behavioral. We believe object-oriented modeling is more appropriate for concurrent development and warrants more investigation.⁷

In practice, the model evolved to accommodate distributed concurrent development, in which multiple development

sites are geographically distributed. (This was necessary primarily because of Tokyo's high density, which forced teams to locate in distant areas.) There are many complex management issues associated with the distributed model, so it is worth investigating technology for collaborating distributed development teams in concurrent development.

Our special interest is which techniques developed for concurrent development can be commonly used in other development projects.

We hope various management techniques developed in the concurrent development system can be applied to other development projects. Future study is necessary from both theory and practice since our management system is still evolving. ♦

REFERENCES

1. M. Aoyama, "Concurrent Development of Software Systems: A New Development Paradigm," *ACM Software Eng. Notes*, July 1987, pp. 20-24.
2. W. Agresti, *New Paradigms for Software Development*, IEEE CS Press, Los Alamitos, Calif., 1986.
3. L. Osterweil, "Software Processes are Software Too," *Proc. Int'l Conf. Software Eng.*, IEEE CS Press, Los Alamitos, Calif., 1987, pp. 2-13.
4. T. Ohno, *Toyota Production System*, Diamond Publications, Tokyo, 1978 (in Japanese; English translation is available from Productivity Press, Cambridge, Mass., 1988).
5. D. Roos et al., *The Machine That Changed the World: The Story of Lean Production*, Harper Perennial, New York, 1990.
6. M. Aoyama et al., "Design Specification in Japan: Tree-Structured Charts," *IEEE Software*, Mar. 1989, pp. 31-37.
7. M. Aoyama, "Distributed Concurrent Development of Software Systems: An Object-Oriented Process Model," *Proc. CompSAC*, IEEE CS Press, Los Alamitos, Calif., 1990, pp. 330-337.
8. L. Putnum, *Software Cost Estimation and Life-Cycle Control*, IEEE CS Press, Los Alamitos, Calif., 1980.
9. M. Cusumano, *Japan's Software Factories*, Oxford University Press, New York, 1991.
10. M. Paulk et al., "Capability Maturity Model for Software," Tech. Report CMU/SEI-91-TR-24, ADA240603, Software Eng. Inst., Pittsburgh, 1991.



Mikio Aoyama is a supervisor at Fujitsu Ltd. His research interests include real-time distributed software systems, software-development environments, and software-process management.

Aoyama received a BS and an MS in electronics from Okayama University. He is a member of the IEEE Computer Society, ACM, Information Processing Society of Japan, and Institute of Electronics, Information, and Communication Engineers, Japan.

Address questions about this article to Aoyama at Fujitsu Ltd., Business Switching Software Div., 2-12-5 Shimo-odnaka, Nakahara-ku, Kawasaki 211 Japan; Internet mikio@miki.nakahara.fujitsu.co.jp.